Time Series Multivariate

FRE6871 & FRE7241, Spring 2026

Jerzy Pawlowski *jp3900@nyu.edu*

NYU Tandon School of Engineering

January 22, 2026

The *Alpha* and *Beta* of Stock Returns

The time series of stock returns $r_i - r_f$ in excess of the risk-free rate r_f, can be decomposed into *systematic* returns $\beta(r_m - r_f)$ (where $r_m - r_f$ is the time series of the excess market returns) plus *idiosyncratic* returns $\alpha + \varepsilon_i$ (which are uncorrelated to the market returns):

$$r_i - r_f = \alpha + \beta(r_m - r_f) + \varepsilon_i$$

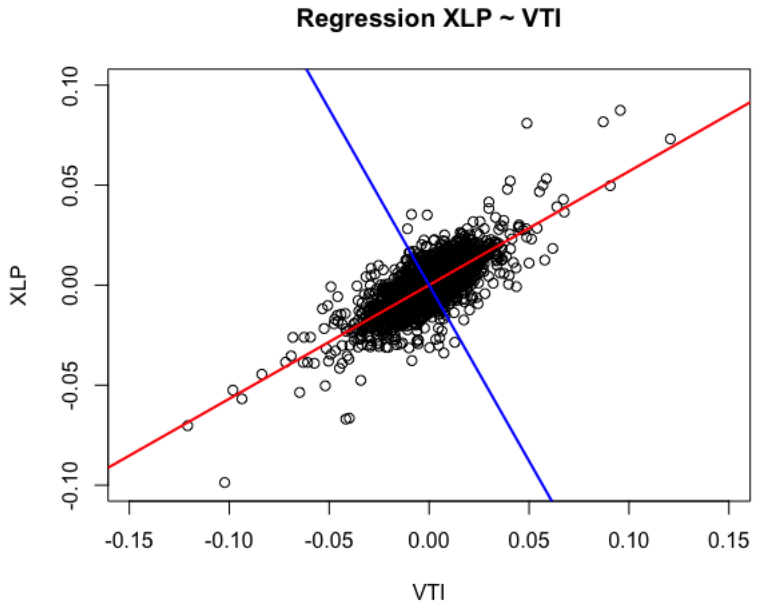
The *alpha* α are the abnormal returns in excess of the risk premium, and ε_i are the regression residuals with zero mean.

The regression residuals ε_i are uncorrelated to the market returns, and can be reduced through portfolio diversification.

The stock β is equal to the ratio of the covariance of the stock returns with the market returns $\sigma_{i,m}$ divided by the variance of the market returns σ_m^2:

$$\beta = \frac{\sigma_{i,m}}{\sigma_m^2} = \rho_{i,m}\frac{\sigma_i}{\sigma_m}$$

Where $\rho_{i,m}$ is the correlation of the stock returns with the market returns, and σ_i and σ_m are the standard deviations of the stock and market returns, respectively.




```
> # Perform regression using formula
> retp <- na.omit(rutils::etfenv$returns[, c("XLP", "VTI")])
> raterf <- 0.03/252
> retp <- (retp - raterf)
> regmod <- lm(XLP ~ VTI, data=retp)
> regsum <- summary(regmod)
> # Get regression coefficients
> coef(regsum)
> # Get alpha and beta
> coef(regsum)[, 1]
> # Plot scatterplot of returns with aspect ratio 1
> plot(XLP ~ VTI, data=rutils::etfenv$returns, main="Regression XLP
+      xlim=c(-0.1, 0.1), ylim=c(-0.1, 0.1), pch=1, col="blue", asp=
> # Add regression line and perpendicular line
> abline(regmod, lwd=2, col="red")
> abline(a=0, b=-1/coef(regsum)[2, 1], lwd=2, col="blue")
```

Capital Asset Pricing Model (*CAPM*)

The *CAPM* model states that the expected return of the stock n: $\mathbb{E}[R_n]$ should be proportional to its beta β_n times the expected excess return of the market $\mathbb{E}[R_m] - r_f$:

$$\mathbb{E}[R_n] = r_f + \beta_n(\mathbb{E}[R_m] - r_f)$$

The *CAPM* model states that stocks should only produce a *systematic* return proportional to their *beta* β values. If a stock has a higher beta then it should also produce higher returns.

The *CAPM* model is not the same as the regression model used to determine the *alpha* α and *beta* β values of stock returns.

In reality, stocks also have a non-zero mean idiosyncratic return *alpha* α_n, in addition to the *systematic* return proportional to their β_n:

$$\mathbb{E}[R_n] = r_f + \alpha_n + \beta_n(\mathbb{E}[R_m] - r_f)$$

```
> library(PerformanceAnalytics)
> # Calculate XLP beta
> PerformanceAnalytics::CAPM.beta(Ra=retp$XLP, Rb=retp$VTI)
> # Or
> betac <- drop(cov(retp$XLP, retp$VTI)/var(retp$VTI))
> betac
> # Calculate XLP alpha
> PerformanceAnalytics::CAPM.alpha(Ra=retp$XLP, Rb=retp$VTI)
> # Or
> alphac <- mean(retp$XLP - betac*retp$VTI)
> # Calculate XLP bull beta
> PerformanceAnalytics::CAPM.beta.bull(Ra=retp$XLP, Rb=retp$VTI)
> # Calculate XLP bear beta
> PerformanceAnalytics::CAPM.beta.bear(Ra=retp$XLP, Rb=retp$VTI)
```

The *Capital Market Line* For a Single Stock

The *CAPM* equation can be expressed in terms of the standard deviation σ_n of the stock returns:

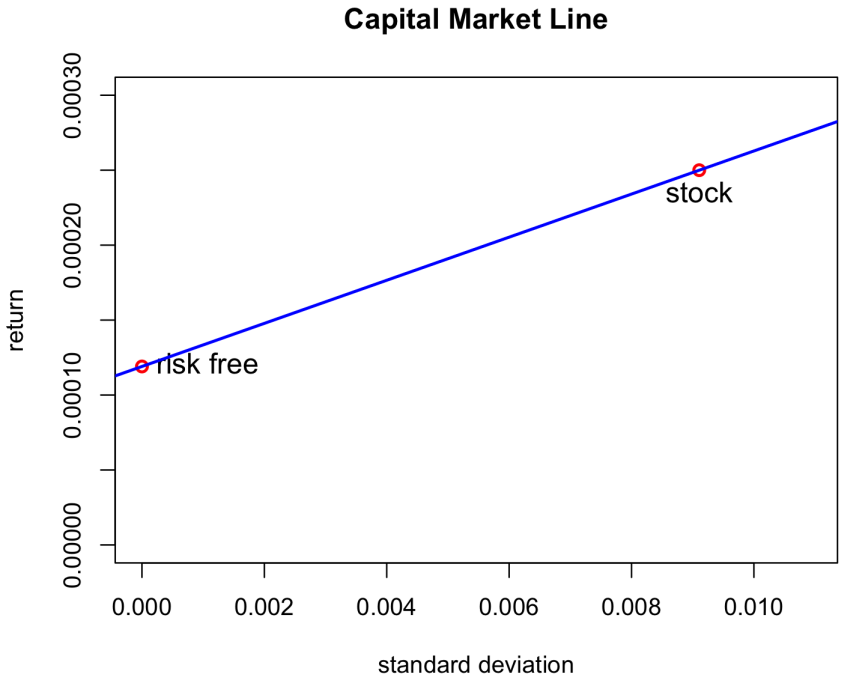
$$\mathbb{E}[R_n] = r_f + \alpha_n + \rho_{n,m} \frac{\sigma_n}{\sigma_m} (\mathbb{E}[R_m] - r_f)$$

This is the *Capital Market Line* (CML), which states that the expected return of the portfolio of the stock and the risk-free asset is proportional to its standard deviation σ_n.

The risk-free asset has zero standard deviation and an expected return equal to the risk-free rate r_f. So when it's added to the stock, the portfolio's return and standard deviation both decrease.

The sum of the portfolio weights is equal to *one*:

$$w_s + w_{rf} = 1$$




```
> # Plot the CML
> rets <- raterf + betac*mean(retp$VTI)
> stdev <- sd(retp$XLP)
> plot(x=c(0, stdev), y=c(raterf, rets), col="red", lwd=2,
+      xlim=c(0, 1.2*stdev), ylim=c(0, 1.2*rets),
+      main="Capital Market Line",
+      xlab="standard deviation", ylab="return")
> text(x=0.0, y=raterf, labels="risk free", pos=4, cex=1.2)
> text(x=stdev, y=rets, labels="stock", pos=1, cex=1.2)
> abline(a=raterf, b=betac*mean(retp$VTI)/stdev, lwd=2, col="blue")
```

The Statistical Significance of *Alpha* and *Beta*

The *t*-statistic (*t*-value) is the ratio of the estimated value divided by its standard error.

The *p*-value is the probability of obtaining values exceeding the *t*-statistic, assuming the *null hypothesis* is true.

A small *p*-value means that the regression coefficients are very unlikely to be zero (given the data).

The *beta* β values of stock returns are very statistically significant, but the *alpha* α values are mostly not significant.

The *p*-value of the *Durbin-Watson* test is large, which indicates that the regression residuals are not autocorrelated.

In practice, the α, β, and the risk-free rate r_f, depend on the time interval of the data, so they're time dependent.

```
> # Get regression coefficients
> coef(regsum)
> # Calculate regression coefficients from scratch
> betac <- drop(cov(retp$XLP, retp$VTI)/var(retp$VTI))
> alphac <- drop(mean(retp$XLP) - betac*mean(retp$VTI))
> c(alphac, betac)
> # Calculate the residuals
> residuals <- (retp$XLP - (alphac + betac*retp$VTI))
> # Calculate the standard deviation of residuals
> nrows <- NROW(residuals)
> residsd <- sqrt(sum(residuals^2)/(nrows - 2))
> # Calculate the standard errors of beta and alpha
> sum2 <- sum((retp$VTI - mean(retp$VTI))^2)
> betasd <- residsd/sqrt(sum2)
> alphasd <- residsd*sqrt(1/nrows + mean(retp$VTI)^2/sum2)
> c(alphasd, betasd)
> # Perform the Durbin-Watson test of autocorrelation of residuals
> lmtest::dwtest(regmod)
```

The *Alpha* and *Beta* of ETF Returns

The *beta* β values of ETF returns are very statistically significant, but the *alpha* α values are mostly not significant.

Some of the ETFs with significant *alpha* α values are the bond ETFs *IEF* and *TLT* (which have performed very well), and the natural resource ETFs *USO* and *DBC* (which have performed very poorly).

```
> retm <- rutils::etfenv$returns
> symbolv <- colnames(retm)
> symbolv <- symbolv[symbolv != "VTI"]
> # Perform regressions and collect statistics
> betam <- sapply(symbolv, function(symbol) {
+ # Specify regression formula
+   formulav <- as.formula(paste(symbol, "~ VTI"))
+ # Perform regression
+   regmod <- lm(formulav, data=retm)
+ # Get regression summary
+   regsum <- summary(regmod)
+ # Collect regression statistics
+   with(regsum,
+     c(beta=coefficients[2, 1],
+ betap=coefficients[2, 4],
+ alpha=coefficients[1, 1],
+ alphap=coefficients[1, 4],
+ dwp=lmtest::dwtest(regmod)$p.value))
+ })  ##  end sapply
> betam <- t(betam)
> # Sort by alphap
> betam <- betam[order(betam[, "alphap"]), ]
```

```
> betam
        beta     betap     alpha   alphap      dwp
IEF  -0.1085 1.41e-125  1.83e-04 0.000621 5.49e-01
VXX  -2.8094  0.00e+00 -1.33e-03 0.000959 5.24e-01
GLD   0.0581  4.09e-06  3.99e-04 0.008808 8.71e-01
USO   0.6830 3.13e-157 -6.85e-04 0.025061 9.34e-02
TLT  -0.2317 1.02e-129  2.47e-04 0.027656 7.24e-01
VEU   0.9747  0.00e+00 -1.98e-04 0.031040 1.00e+00
XLF   1.2479  0.00e+00 -2.30e-04 0.051930 1.00e+00
AIEQ  1.1541 5.26e-230 -2.85e-04 0.150440 9.04e-01
IVE   0.9679  0.00e+00 -6.04e-05 0.204552 1.00e+00
EEM   1.1718  0.00e+00 -1.52e-04 0.242707 1.00e+00
VNQ   1.1367  0.00e+00 -1.87e-04 0.243060 1.00e+00
SVXY  2.1154 1.79e-244 -7.36e-04 0.246519 1.58e-07
XLP   0.5473  0.00e+00  8.98e-05 0.259713 1.00e+00
IWD   0.9652  0.00e+00 -4.78e-05 0.303134 1.00e+00
USMV  0.7043  0.00e+00  5.79e-05 0.371531 9.16e-01
VLUE  0.9686  0.00e+00 -7.33e-05 0.396673 9.86e-01
XLV   0.7276  0.00e+00  6.85e-05 0.414988 7.87e-01
DBC   0.4000 1.01e-199 -1.26e-04 0.421492 9.40e-01
IVW   0.9833  0.00e+00  3.35e-05 0.426340 1.00e+00
QQQ   1.0973  0.00e+00  5.86e-05 0.484433 9.98e-01
XLE   1.0782  0.00e+00 -1.10e-04 0.502282 4.12e-01
DYNF  0.9705  0.00e+00  3.97e-05 0.510397 9.70e-01
XLU   0.6348  0.00e+00  6.75e-05 0.569779 1.00e+00
QUAL  0.9672  0.00e+00  2.13e-05 0.591942 9.98e-01
XLB   1.0196  0.00e+00 -4.42e-05 0.653989 1.00e+00
VTV   0.9377  0.00e+00 -2.25e-05 0.680647 1.00e+00
XLK   1.1113  0.00e+00  3.02e-05 0.722612 9.99e-01
IWF   1.0116  0.00e+00  2.20e-05 0.739039 1.00e+00
XLY   1.0432  0.00e+00  2.32e-05 0.763740 1.00e+00
SPY   0.9859  0.00e+00 -5.71e-06 0.787707 1.00e+00
MTUM  1.0134  0.00e+00  2.37e-05 0.801167 3.47e-02
XLI   0.9964  0.00e+00 -1.05e-05 0.880625 1.00e+00
IWB   0.9829  0.00e+00 -2.43e-06 0.898327 1.00e+00
VYM   0.8572  0.00e+00  9.89e-07 0.988337 1.00e+00
```

The Security Market Line for ETFs

The *Security Market Line* (SML) represents the linear relationship between expected stock returns and their *systematic* risk β.

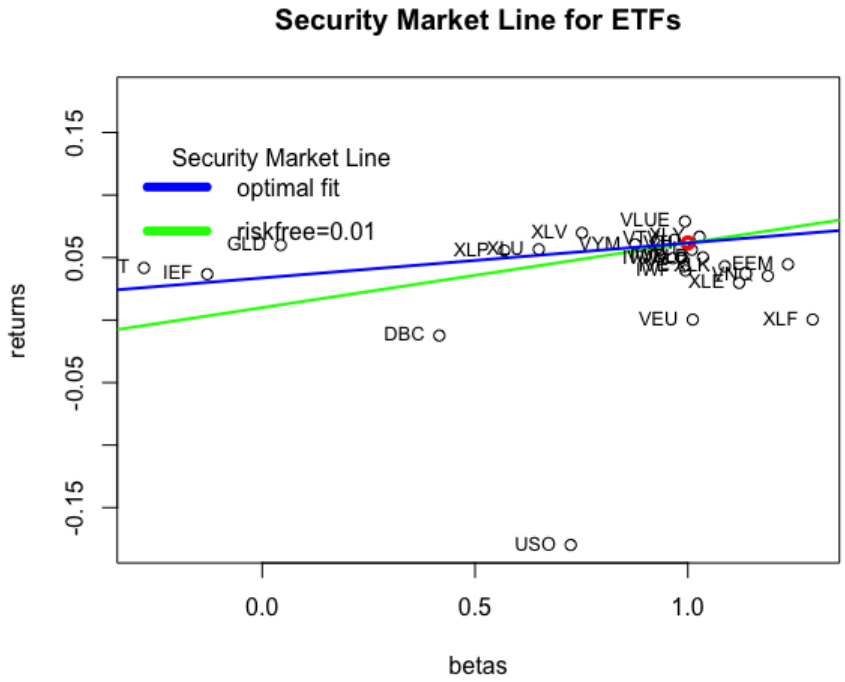
The *SML* depends on the choice of the risk-free rate r_f, with a steeper *SML* line for lower risk-free rates r_f.

All the different *SML* lines pass through the point $(\beta = 1, r = R_m)$ corresponding to the market, and they intersect the y-axis at the risk-free point $(\beta = 0, r = r_f)$.

A scatterplot of asset returns versus their β shows which assets earn a positive α, and which don't.

If an asset lies on the *SML*, then its returns are mostly *systematic*, and its α is equal to zero.

Assets above the *SML* have a positive α, and those below have a negative α.




```
> symbolv <- rownames(betam)
> betac <- betam[-match(c("VXX", "SVXY", "MTUM", "USMV", "QUAL"), s
> betac <- c(1, betac)
> names(betac)[1] <- "VTI"
> retsann <- sapply(retp[, names(betac)], PerformanceAnalytics::Ret
> # Plot scatterplot of returns vs betas
> minrets <- min(retsann)
> plot(retsann ~ betac, xlab="betas", ylab="returns",
+      ylim=c(minrets, -minrets), main="Security Market Line for E
> retvti <- retsann["VTI"]
> points(x=1, y=retvti, col="red", lwd=3, pch=21)
> # Plot Security Market Line
> raterf <- 0.01
```

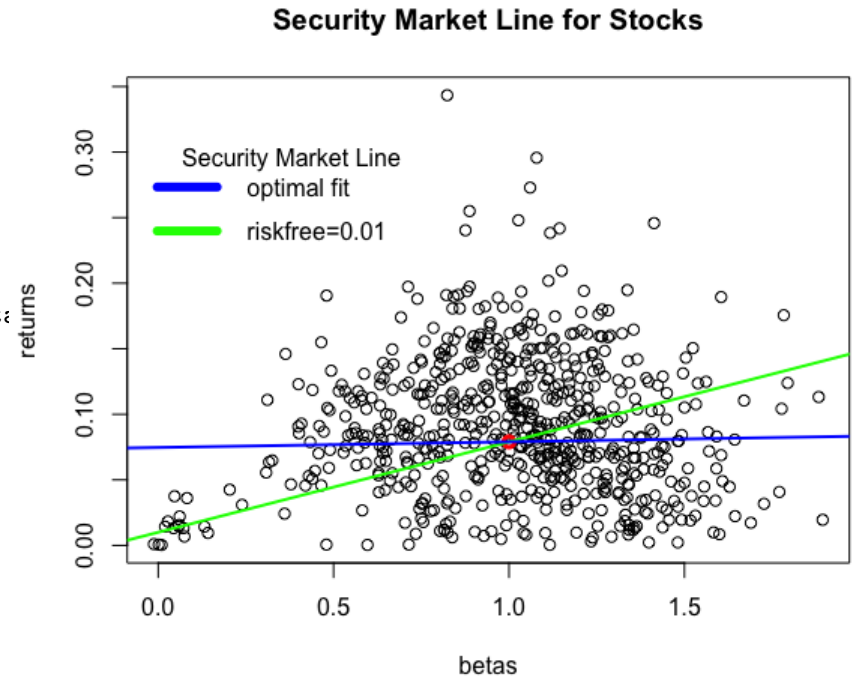
```
> # Add labels
> text(x=betac, y=retsann, labels=names(betac), pos=2, cex=0.8)
> # Find optimal risk-free rate by minimizing residuals
> rss <- function(raterf) {
+   sum((retsann - raterf - betac*(retvti-raterf))^2)
+ }  ##  end rss
> optimrss <- optimize(rss, c(-1, 1))
> raterf <- optimrss$minimum
> # Or simply
> retsadj <- (retsann - retvti*betac)
> betadj <- (1-betac)
> raterf <- sum(retsadj*betadj)/sum(betadj^2)
> abline(a=raterf, b=(retvti-raterf), col="blue", lwd=2)
> legend(x="topleft", bty="n", title="Security Market Line",
+  legend=c("optimal fit", "raterf=0.01"),
+  y.intersp=0.5, cex=1.0, lwd=6, lty=1, col=c("blue", "green"))
```

The Security Market Line for Stocks

The best fitting *Security Market Line* (SML) for stocks is almost flat, which shows that stocks with higher β don't earn higher returns.

This is called the *low beta anomaly*.

```
> # Load S&P500 constituent stock returns
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RDat
> retvti <- na.omit(rutils::etfenv$returns$VTI)
> retp <- retstock[index(retvti), ]
> nrows <- NROW(retp)
> # Calculate stock betas
> betac <- sapply(retp, function(x) {
+   retp <- na.omit(cbind(x, retvti))
+   drop(cov(retp[, 1], retp[, 2])/var(retp[, 2]))
+ })  ##  end sapply
> mean(betac)
> # Calculate annual stock returns
> retsann <- retp
> retsann[1, ] <- 0
> retsann <- zoo::na.locf(retsann, na.rm=FALSE)
> retsann <- 252*sapply(retsann, sum)/nrows
> # Remove stocks with zero returns
> sum(retsann == 0)
> betac <- betac[retsann > 0]
> retsann <- retsann[retsann > 0]
> retvti <- 252*mean(retvti)
> # Plot scatterplot of returns vs betas
> plot(retsann ~ betac, xlab="betas", ylab="returns",
+      main="Security Market Line for Stocks")
> points(x=1, y=retvti, col="red", lwd=3, pch=21)
> # Plot Security Market Line
> raterf <- 0.01
> abline(a=raterf, b=(retvti-raterf), col="green", lwd=2)
```




```
> # Find optimal risk-free rate by minimizing residuals
> retsadj <- (retsann - retvti*betac)
> betadj <- (1-betac)
> raterf <- sum(retsadj*betadj)/sum(betadj^2)
> abline(a=raterf, b=(retvti-raterf), col="blue", lwd=2)
> legend(x="topleft", bty="n", title="Security Market Line",
+  legend=c("optimal fit", "raterf=0.01"),
+  y.intersp=0.5, cex=1.0, lwd=6, lty=1, col=c("blue", "green"))
```

The Capital Market Line And The Security Market Line

The CAPM formula can be applied in three different contexts.

First, as a regression of the time series of stock returns versus the time series of market returns, to determine the alpha α and beta β coefficients:

$$r_i = r_f + \alpha + \beta(r_m - r_f) + \varepsilon_i$$

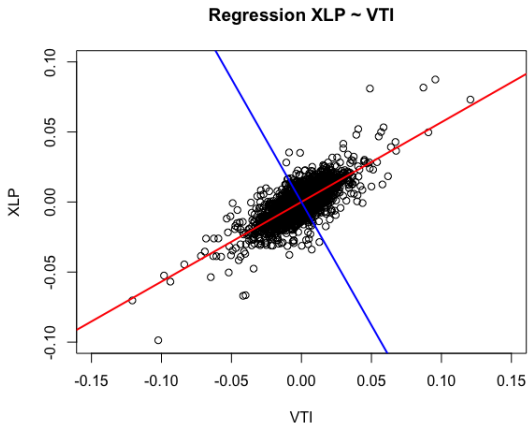
Second, as the Security Market Line (SML), which relates the expected stock returns to their betas β_n:

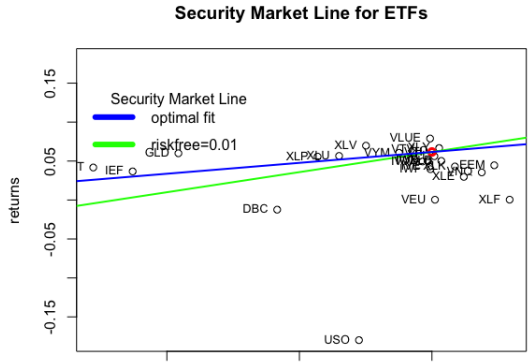
$$\mathbb{E}[R_n] = r_f + \alpha_n + \beta_n(\mathbb{E}[R_m] - r_f)$$

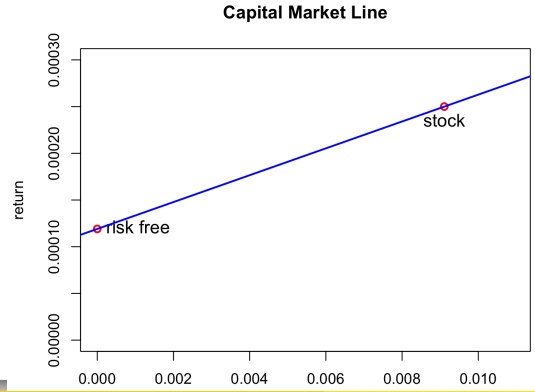
Third, as the Capital Market Line (CML), which relates the return of the portfolio of a stock and the risk-free asset to its standard deviation σ_n.

$$\mathbb{E}[R_n] = r_f + \alpha_n + \rho_{n,m}\frac{\sigma_n}{\sigma_m}(\mathbb{E}[R_m] - r_f)$$

Note that the risk-free asset has zero standard deviation, and an expected return equal to r_f.







The Treynor and Information Ratios

The *Treynor* ratio measures the excess returns per unit of the *systematic* risk *beta* β, and is equal to the excess returns (over a risk-free rate) divided by the β:

$$T_r = \frac{E[R - r_f]}{\beta}$$

The *Treynor* ratio is similar to the *Sharpe* ratio, with the difference that its denominator represents only *systematic* risk, not total risk.

The *Information* ratio is equal to the excess returns (over a benchmark) divided by the *tracking error* (standard deviation of excess returns):

$$I_r = \frac{E[R - R_b]}{\sqrt{\sum_{i=1}^{n}(R_i - R_{i,b})^2}}$$

The *Information* ratio measures the amount of outperformance versus the benchmark, and the consistency of outperformance.

```
> retp <- na.omit(rutils::etfenv$returns[, c("XLP", "VTI")])
> library(PerformanceAnalytics)
> # Calculate XLP Treynor ratio
> TreynorRatio(Ra=retp$XLP, Rb=retp$VTI)
[1] 0.116
> # Calculate XLP Information ratio
> InformationRatio(Ra=retp$XLP, Rb=retp$VTI)
[1] -0.0833
```

CAPM Summary Statistics

`PerformanceAnalytics::table.CAPM()` calculates the *beta* β and *alpha* α values, the *Treynor* ratio, and other performance statistics.

```
> PerformanceAnalytics::table.CAPM(Ra=retm[, c("XLP", "XLF")],
+                           Rb=retm$VTI, scale=252)
                   XLP to VTI XLF to VTI
Alpha                  0.0001    -0.0002
Beta                   0.5473     1.2479
Alpha Robust           0.0001    -0.0003
Beta Robust            0.5450     1.0756
Beta+                  0.5626     1.3164
Beta-                  0.5636     1.3227
Beta+ Robust           0.5444     1.0934
Beta- Robust           0.5516     1.1138
R-squared              0.5263     0.7233
R-squared Robust       0.4249     0.6194
Annualized Alpha       0.0229    -0.0564
Correlation            0.7255     0.8505
Correlation p-value    0.0000     0.0000
Tracking Error         0.1320     0.1553
Active Premium        -0.0110    -0.0595
Information Ratio     -0.0833    -0.3828
Treynor Ratio          0.1001     0.0149
```

```
> capmstats <- table.CAPM(Ra=retm[, symbolv], Rb=retm$VTI, scale=252)
> colv <- strsplit(colnames(capmstats), split=" ")
> colv <- do.call(cbind, colv)[1, ]
> colnames(capmstats) <- colv
> capmstats <- t(capmstats)
> capmstats <- capmstats[, -1]
> colv <- colnames(capmstats)
> whichv <- match(c("Annualized Alpha", "Information Ratio", "Treynor Rat
> colv[whichv] <- c("Alpha", "Information", "Treynor")
> colnames(capmstats) <- colv
> capmstats <- capmstats[order(capmstats[, "Alpha"], decreasing=TRUE), ]
> # Copy capmstats into etfenv and save to .RData file
> etfenv <- rutils::etfenv
> etfenv$capmstats <- capmstats
```

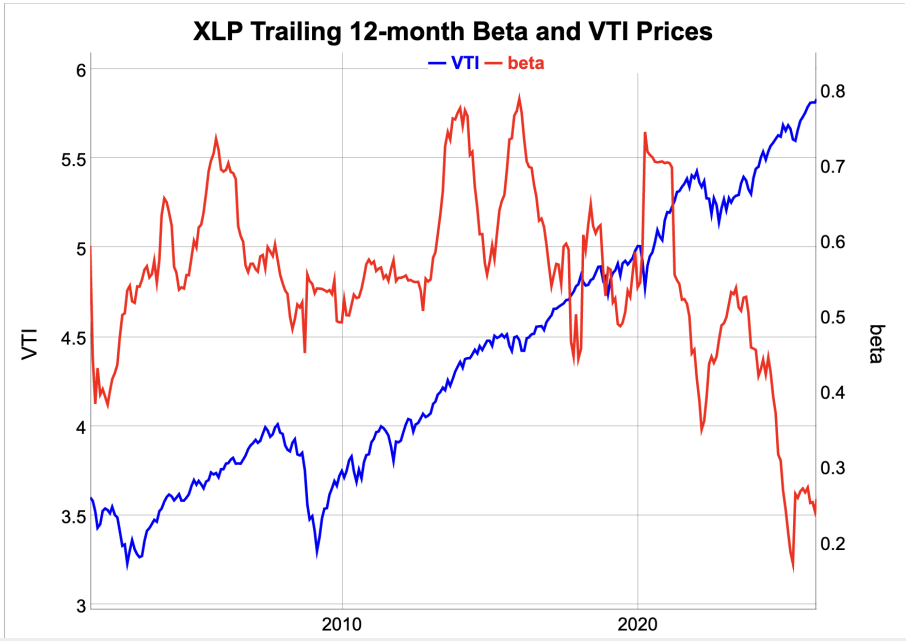
```
> rutils::etfenv$capmstats[, c("Beta", "Alpha", "Information"
        Beta   Alpha Information Treynor
GLD   0.0515  0.1066      0.0301  1.8482
TLT  -0.2350  0.0641     -0.2470 -0.1144
IEF  -0.1101  0.0471     -0.2791 -0.2940
XLU   0.6116  0.0246     -0.0473  0.0938
XLV   0.7332  0.0224      0.0221  0.0930
XLP   0.5433  0.0208     -0.0746  0.1009
XLY   1.0126  0.0120      0.0518  0.0706
XLI   0.9657  0.0116      0.0535  0.0733
USMV  0.7231  0.0102     -0.3505  0.1489
QQQ   1.1895  0.0040      0.0106  0.0546
VTI   0.9948  0.0032      0.1023  0.0749
IVW   0.9955  0.0025      0.0114  0.0654
IWB   0.9786  0.0022      0.0185  0.0655
XLB   0.9443  0.0021     -0.0825  0.0563
DYNF  0.9939  0.0014     -0.0008  0.1343
MTUM  1.0322  0.0000     -0.0030  0.1241
IWD   0.9422 -0.0005     -0.0852  0.0624
VYM   0.8633 -0.0008     -0.1715  0.0839
QUAL  0.9889 -0.0013     -0.1010  0.1206
XLE   0.9806 -0.0024     -0.1313  0.0377
XLK   1.1601 -0.0026     -0.0326  0.0527
IWF   1.0323 -0.0030     -0.0683  0.0565
IVE   0.9551 -0.0047     -0.1449  0.0577
VTV   0.9440 -0.0056     -0.1879  0.0787
VLUE  0.9790 -0.0229     -0.3956  0.0985
DBC   0.4003 -0.0316     -0.4631 -0.0228
EEM   1.1803 -0.0362     -0.2442  0.0479
XLF   1.2158 -0.0405     -0.2882  0.0152
VNQ   1.1326 -0.0449     -0.3073  0.0276
VEU   0.9861 -0.0505     -0.6122  0.0266
AIEQ  1.1493 -0.0712     -0.7815  0.1116
USO   0.6903 -0.1598     -0.7079 -0.2287
SVXY  2.1465 -0.1774         NaN     NaN
VXX  -2.8726 -0.2765     -0.9328  0.2135
```

Trailing Stock Beta Over Time

The trailing beta of *XLP* versus *VTI* changes over time, with lower beta in periods of stock selloffs.

The function `roll_reg()` from package *HighFreq* performs trailing regressions in `C++` (*RcppArmadillo*), so it's therefore much faster than equivalent `R` code.

```
> # Calculate XLP and VTI returns
> retp <- na.omit(rutils::etfenv$returns[, c("XLP", "VTI")])
> # Calculate monthly end points
> endd <- rutils::calc_endpoints(retp, interval="months")[-1]
> # Calculate start points from look-back interval
> lookb <- 12  ##  Look back 12 months
> startp <- c(rep(1, lookb), endd[1:(NROW(endd)-lookb)])
> head(cbind(endd, startp), lookb+2)
> # Calculate trailing beta regressions every month in R
> formulav <- XLP ~ VTI  ##  Specify regression formula
> betar <- sapply(1:NROW(endd), FUN=function(tday) {
+     datav <- retp[startp[tday]:endd[tday], ]
+     ##  coef(lm(formulav, data=datav))[2]
+     drop(cov(datav$XLP, datav$VTI)/var(datav$VTI))
+ })  ##  end sapply
> # Calculate trailing betas using RcppArmadillo
> controll <- HighFreq::param_reg()
> reg_stats <- HighFreq::roll_reg(respv=retp$XLP, predm=retp$VTI,
+   startp=(startp-1), endd=(endd-1), controll=controll)
> betac <- reg_stats[, 1]
> all.equal(betac, betar)
> # Compare the speed of RcppArmadillo with R code
> library(microbenchmark)
> summary(microbenchmark(
+   Rcpp=HighFreq::roll_reg(respv=retp$XLP, predm=retp$VTI, startp=(startp-1), endd=(endd-1), controll=controll),
+   Rcode=sapply(1:NROW(endd), FUN=function(tday) {
+     datav <- retp[startp[tday]:endd[tday], ]
+     drop(cov(datav$XLP, datav$VTI)/var(datav$VTI))
+   }),
```




```
> # dygraph plot of trailing XLP beta and VTI prices
> datev <- zoo::index(retp[endd, ])
> pricev <- log(rutils::etfenv$prices$VTI[datev])
> datav <- cbind(pricev, betac)
> colnames(datav)[2] <- "beta"
> colv <- colnames(datav)
> dygraphs::dygraph(datav, main="XLP Trailing 12-month Beta and VTI
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="blue", strokeWidth=2) %>%
+   dySeries(name=colv[2], axis="y2", col="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Trailing EMA Stock Beta

The trailing beta β of a stock with returns r_t with respect to a stock index with returns R_t can be updated using these recursive EMA formulas with the decay factor λ:

$$\bar{r}_t = \lambda \bar{r}_{t-1} + (1-\lambda) r_t$$

$$\bar{R}_t = \lambda \bar{R}_{t-1} + (1-\lambda) R_t$$

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1-\lambda)(R_t - \bar{R}_t)^2$$

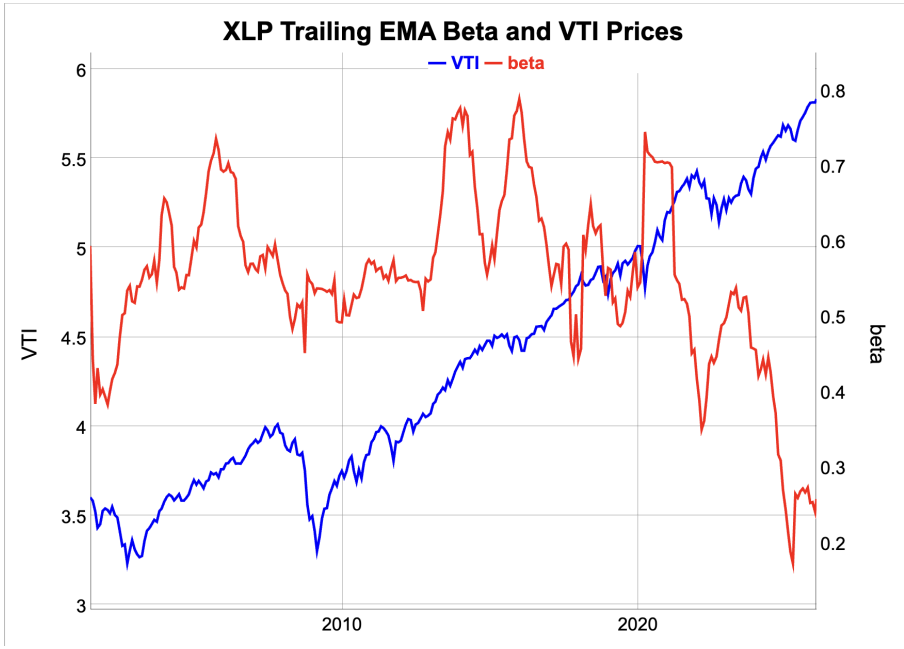
$$\text{cov}_t = \lambda \, \text{cov}_{t-1} + (1-\lambda)(r_t - \bar{r}_t)(R_t - \bar{R}_t)$$

$$\beta_t = \frac{\text{cov}_t}{\sigma_t^2}$$

The parameter λ determines the rate of decay of the weight of past returns. If λ is close to 1 then the decay is weak and past returns have a greater weight, and the trailing mean values have a stronger dependence on past returns. This is equivalent to a long look-back interval. And vice versa if λ is close to 0.

The function `HighFreq::run_covar()` calculates the trailing variances, covariances, and means of two *time series*.

```
> # Calculate the trailing betas
> lambdaf <- 0.99
> covarv <- HighFreq::run_covar(retp, lambdaf)
> betac <- covarv[, 1]/covarv[, 3]
```




```
> # dygraph plot of trailing XLP beta and VTI prices
> datav <- cbind(pricev, betac[endd])[-(1:11)] ##  Remove warmup per
> colnames(datav)[2] <- "beta"
> colv <- colnames(datav)
> dygraphs::dygraph(datav, main="XLP Trailing EMA Beta and VTI Price
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="blue", strokeWidth=2) %>%
+   dySeries(name=colv[2], axis="y2", col="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

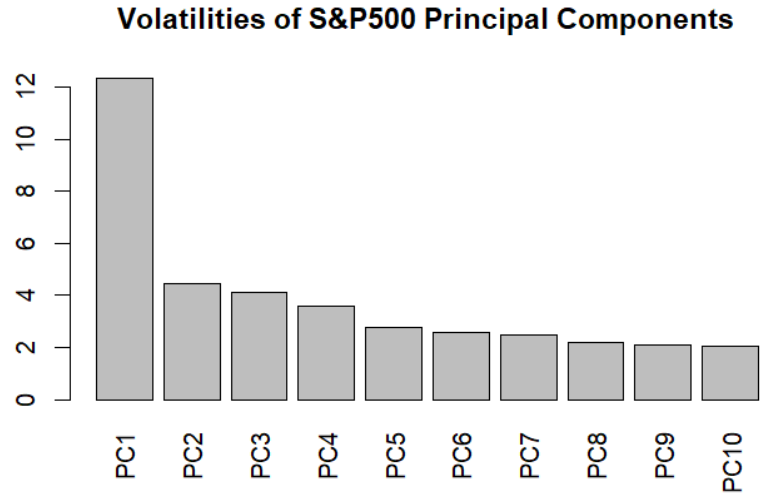
Principal Components of *S&P500* Stock Constituents

The *PCA* standard deviations are the volatilities of the *principal component* time series.

The original time series of returns can be calculated approximately from the first few *principal components* with the largest standard deviations.

The *Kaiser-Guttman* rule uses only *principal components* with *variance* greater than 1.

Another rule of thumb is to use the *principal components* with the largest standard deviations which sum up to 80% of the total variance of returns.




```
> # Load S&P500 constituent stock prices
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_prices.RData'
> # Calculate stock prices and percentage returns
> pricets <- zoo::na.locf(pricets, na.rm=FALSE)
> pricets <- zoo::na.locf(pricets, fromLast=TRUE)
> retp <- rutils::diffit(log(pricev))
> # Standardize (center and scale) the returns
> retp <- lapply(retp, function(x) {(x - mean(x))/sd(x)})
> retp <- rutils::do_call(cbind, retp)
> # Perform principal component analysis PCA
> pcad <- prcomp(retp, scale=TRUE)
> # Find number of components with variance greater than 2
> ncomp <- which(pcad$sdev^2 < 2)[1]
```

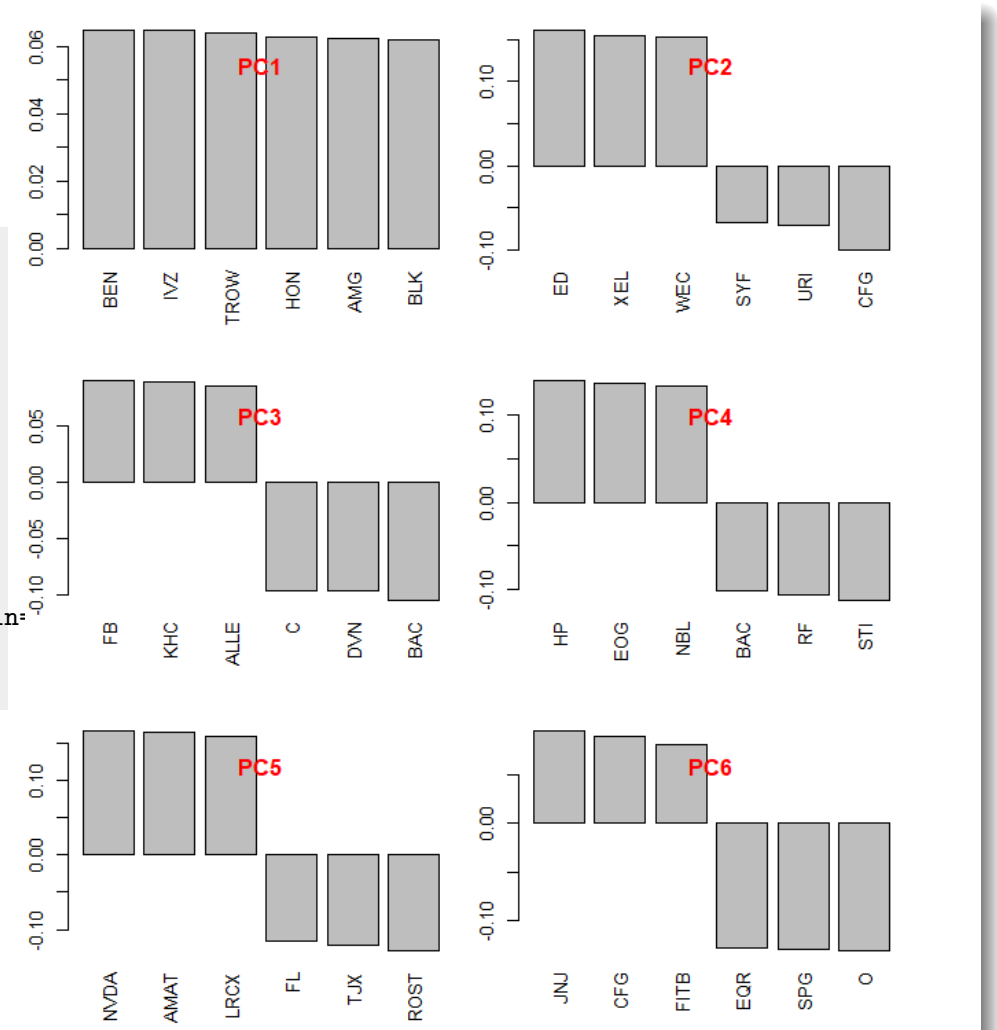
```
> # Plot standard deviations of principal components
> barplot(pcad$sdev[1:ncomp],
+   names.arg=colnames(pcad$rotation[, 1:ncomp]),
+   las=3, xlab="", ylab="",
+   main="Volatilities of S&P500 Principal Components")
```

S&P500 Principal Component Loadings

Principal component loadings are the weights of *principal component* portfolios.

The *principal component* portfolios have mutually orthogonal returns represent the different orthogonal modes of the return variance.

```
> # Calculate principal component loadings (weights)
> # Plot barplots with PCA weights in multiple panels
> ncomps <- 6
> par(mfrow=c(ncomps/2, 2))
> par(mar=c(4, 2, 2, 1), oma=c(0, 0, 0, 0))
> # First principal component weights
> weightv <- sort(pcad$rotation[, 1], decreasing=TRUE)
> barplot(weightv[1:6], las=3, xlab="", ylab="", main="")
> title(paste0("PC", 1), line=-2.0, col.main="red")
> for (ordern in 2:ncomps) {
+   weightv <- sort(pcad$rotation[, ordern], decreasing=TRUE)
+   barplot(weightv[c(1:3, 498:500)], las=3, xlab="", ylab="", main=
+   title(paste0("PC", ordern), line=-2.0, col.main="red")
+ }  ##  end for
```

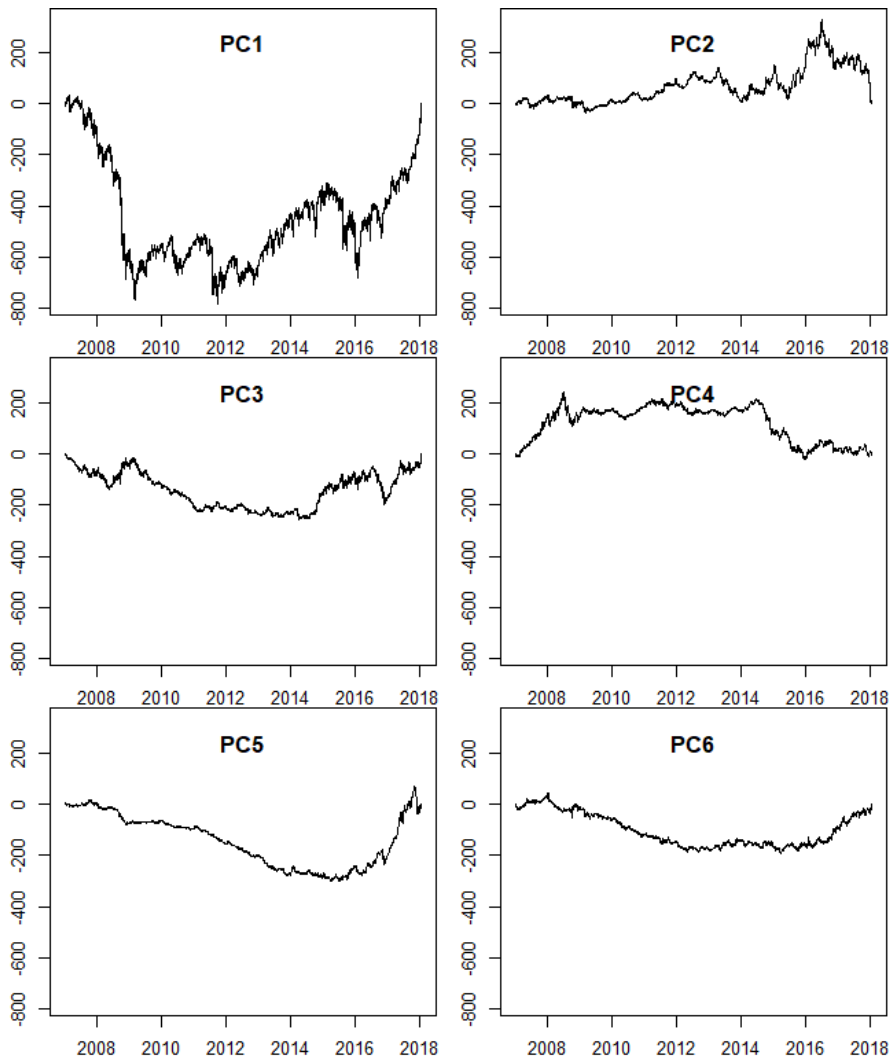


S&P500 Principal Component Time Series

The time series of the *principal components* can be calculated by multiplying the loadings (weights) times the original data.

Higher order *principal components* are gradually less volatile.

```
> # Calculate principal component time series
> retpca <- xts(retp %*% pcad$rotation[, 1:ncomps], order.by=datev)
> round(cov(retpca), 3)
> retpcac <- cumsum(retpca)
> # Plot principal component time series in multiple panels
> par(mfrow=c(ncomps/2, 2))
> par(mar=c(2, 2, 0, 1), oma=c(0, 0, 0, 0))
> rangev <- range(retpcac)
> for (ordern in 1:ncomps) {
+   plot.zoo(retpcac[, ordern], ylim=rangev, xlab="", ylab="")
+   title(paste0("PC", ordern), line=-2.0)
+ }  ##  end for
```



S&P500 Factor Model From *Principal Components*

By inverting the *PCA* analysis, the *S&P500* constituent returns can be calculated from the first `k` *principal components* under a *factor model*:

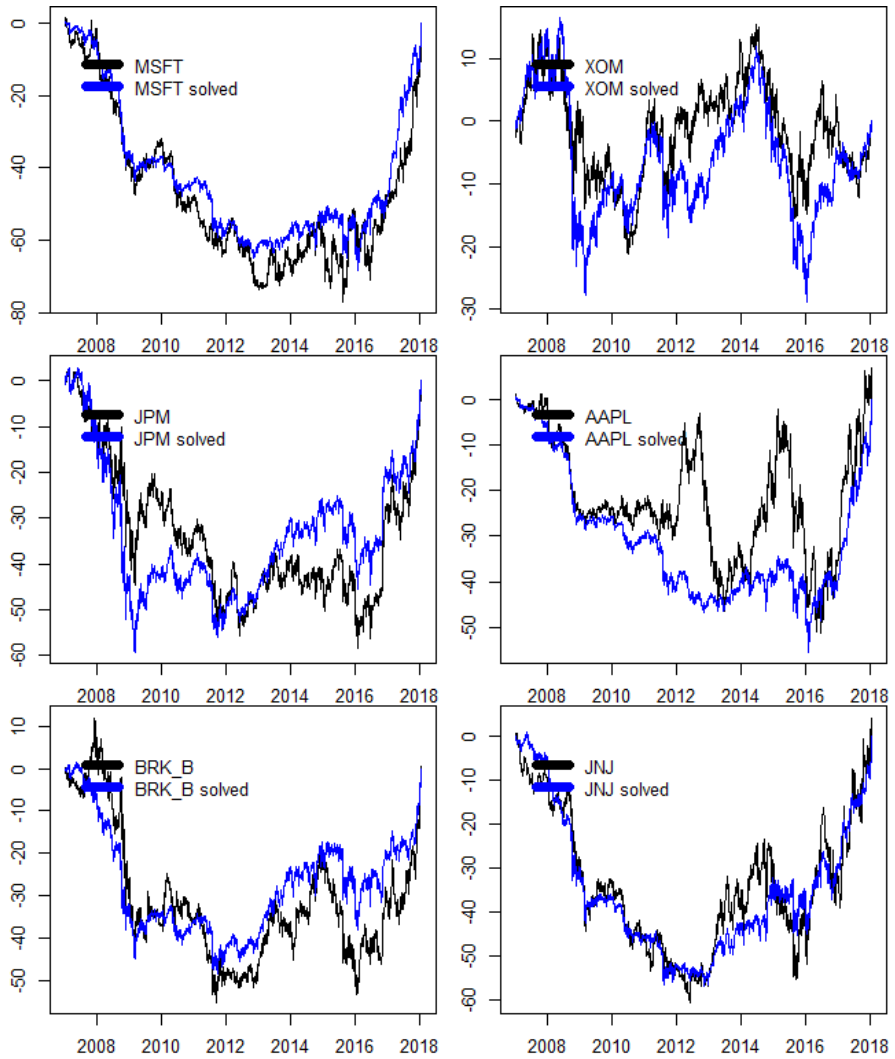
$$\mathbf{r}_i = \alpha_i + \sum_{j=1}^{k} \beta_{ji} \mathbf{F}_j + \varepsilon_i$$

The *principal components* are interpreted as *market factors*: $\mathbf{F}_j = \mathbf{pc}_j$.

The *market betas* are the inverse of the *principal component loadings*: $\beta_{ji} = w_{ij}$.

The ε_i are the *idiosyncratic* returns, which should be mutually independent and uncorrelated to the *market factor* returns.

```
> # Invert principal component time series
> pcinv <- solve(pcad$rotation)
> all.equal(pcinv, t(pcad$rotation))
> solved <- retpca %*% pcinv[1:ncomps, ]
> solved <- xts::xts(solved, datev)
> solved <- cumsum(solved)
> retc <- cumsum(retp)
> # Plot the solved returns
> symbolv <- c("MSFT", "XOM", "JPM", "AAPL", "BRK_B", "JNJ")
> for (symbol in symbolv) {
+   plot.zoo(cbind(retc[, symbol], solved[, symbol]),
+     plot.type="single", col=c("black", "blue"), xlab="", ylab="")
+   legend(x="topleft", bty="n",
+    legend=paste0(symbol, c("", " solved")),
+    title=NULL, inset=0.05, cex=1.0, lwd=6,
+    lty=1, col=c("black", "blue"))
+ }  ##  end for
```



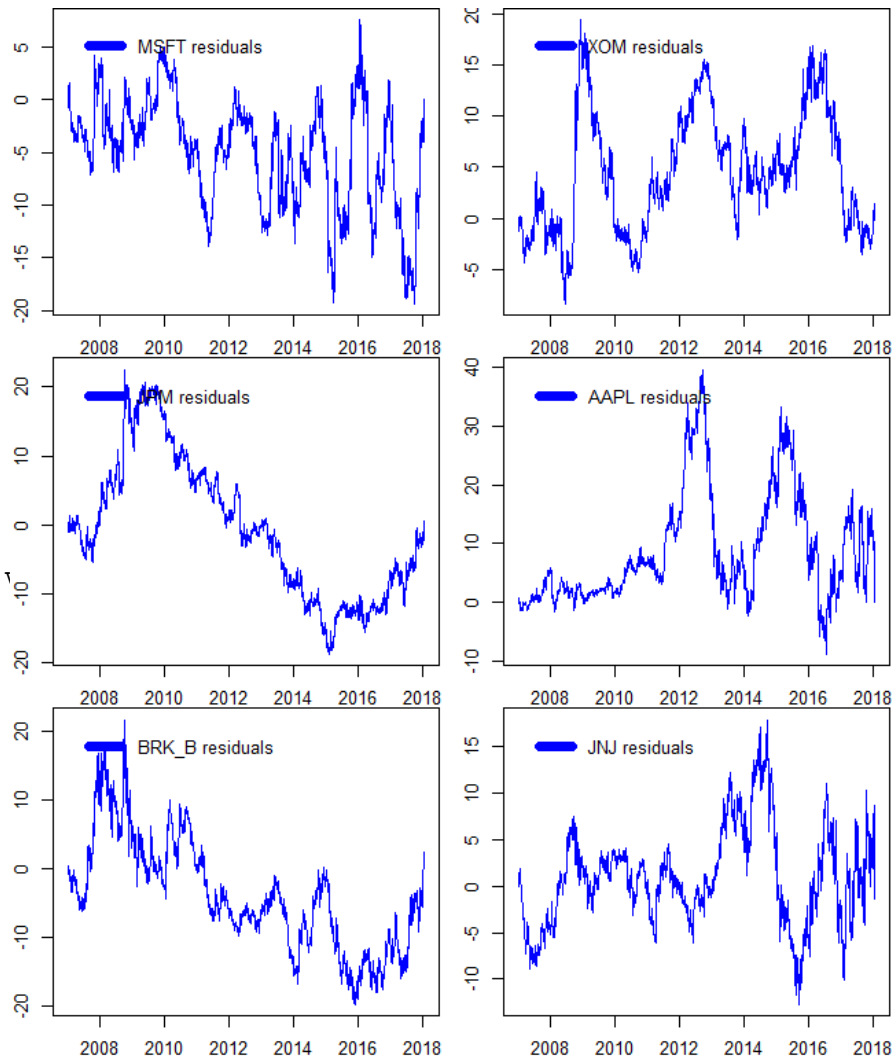
S&P500 Factor Model Residuals

The original time series of returns can be calculated exactly from the time series of all the *principal components*, by inverting the loadings matrix.

The original time series of returns can be calculated approximately from just the first few *principal components*, which demonstrates that *PCA* is a form of *dimension reduction*.

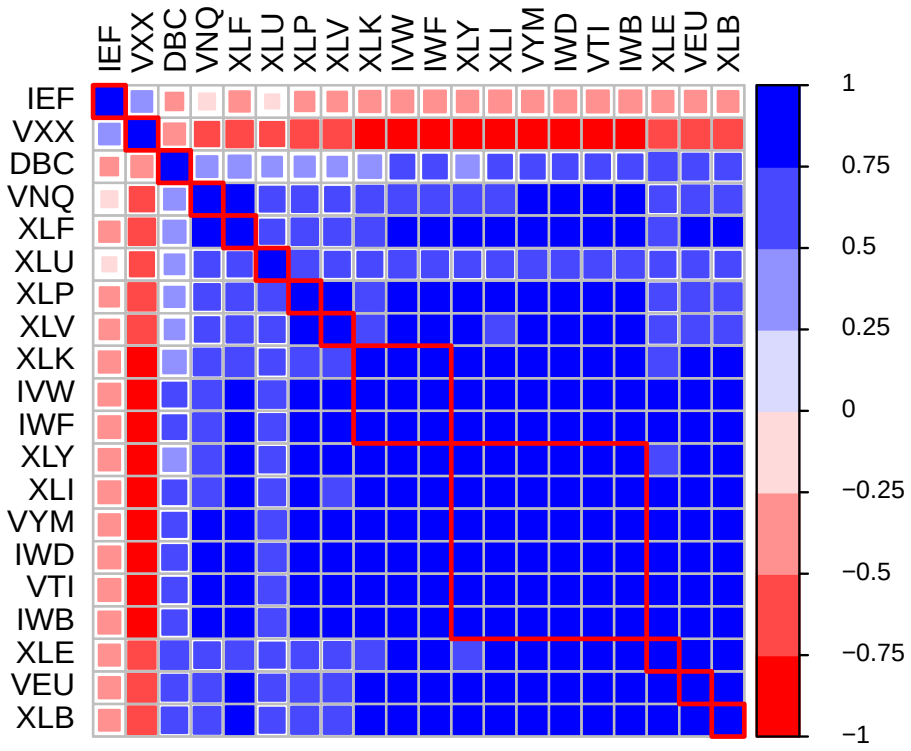
The function `solve()` solves systems of linear equations, and also inverts square matrices.

```
> # Perform ADF unit root tests on original series and residuals
> sapply(symbolv, function(symbol) {
+   c(series=tseries::adf.test(retc[, symbol])$p.value,
+     resid=tseries::adf.test(retc[, symbol] - solved[, symbol])$p.v
+ })  ##  end sapply
> # Plot the residuals
> for (symbol in symbolv) {
+   plot.zoo(retc[, symbol] - solved[, symbol],
+     plot.type="single", col="blue", xlab="", ylab="")
+   legend(x="topleft", bty="n", legend=paste(symbol, "residuals"),
+    title=NULL, inset=0.05, cex=1.0, lwd=6, lty=1, col="blue")
+ }  ##  end for
> # Perform ADF unit root test on principal component time series
> retpca <- xts(retp %*% pcad$rotation, order.by=datev)
> retpcac <- cumsum(retpca)
> adf_pvalues <- sapply(1:NCOL(retpcac), function(ordern)
+   tseries::adf.test(retpcac[, ordern])$p.value)
> # AdF unit root test on stationary time series
> tseries::adf.test(rnorm(1e5))
```



Correlation and Factor Analysis

```
> ### Perform pair-wise correlation analysis
> # Calculate correlation matrix
> cormat <- cor(retp)
> colnames(cormat) <- colnames(retp)
> rownames(cormat) <- colnames(retp)
> # Reorder correlation matrix based on clusters
> # Calculate permutation vector
> library(corrplot)
> ordern <- corrMatOrder(cormat, order="hclust",
+         hclust.method="complete")
> # Apply permutation vector
> cormat <- cormat[ordern, ordern]
> # Plot the correlation matrix
> colorv <- colorRampPalette(c("red", "white", "blue"))
> corrplot(cormat, tl.col="black", tl.cex=0.8,
+     method="square", col=colorv(8),
+     cl.offset=0.75, cl.cex=0.7,
+     cl.align.text="l", cl.ratio=0.25)
> # draw rectangles on the correlation matrix plot
> corrRect.hclust(cormat, k=NROW(cormat) %/% 2,
+           method="complete", col="red")
```



Hierarchical Clustering Analysis

The function `as.dist()` converts a matrix representing the *distance* (dissimilarity) between elements, into a list of class `"dist"`.

For example, `as.dist()` converts `(1-correlation)` to distance.

The function `hclust()` recursively combines elements into clusters based on their mutual *distance*.

First `hclust()` combines individual elements that are closest to each other.

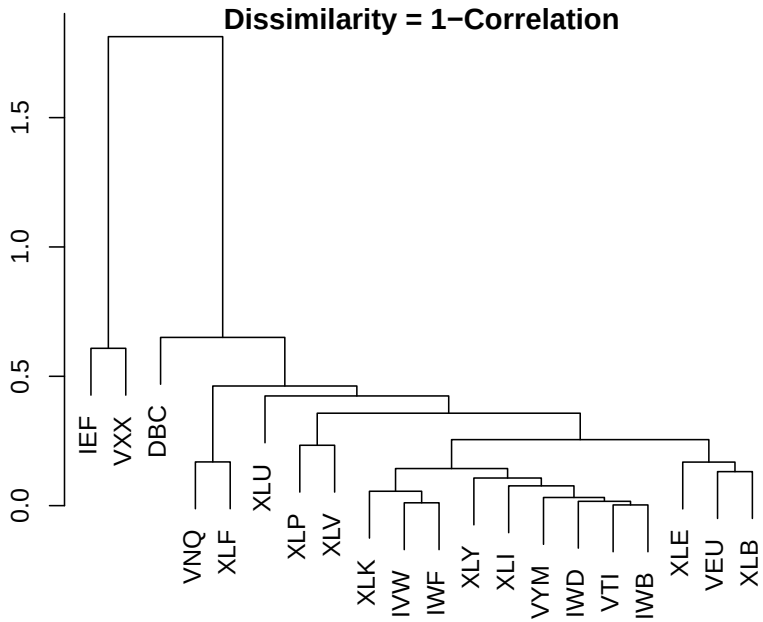
Then it combines elements to the closest clusters, then clusters with other clusters, until all elements are combined into one cluster.

This process of recursive clustering can be represented as a *dendrogram* (tree diagram).

Branches of a *dendrogram* represent clusters.

Neighboring branches contain elements that are close to each other (have small distance).

Neighboring branches combine into larger branches, that then combine with their closest branches, etc.

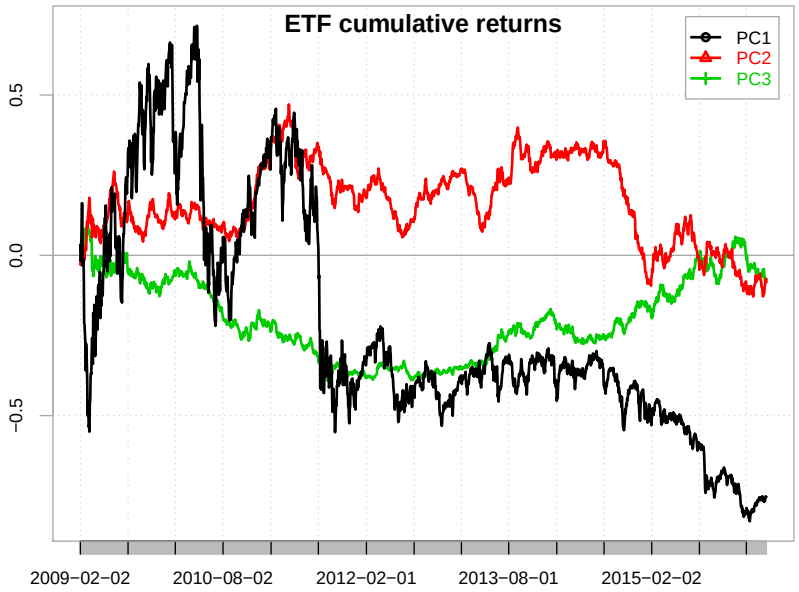



```
> # Convert correlation matrix into distance object
> distancev <- as.dist(1-cormat)
> # Perform hierarchical clustering analysis
> compclust <- hclust(distancev)
> plot(compclust, ann=FALSE, xlab="", ylab="")
> title("Dendrogram representing hierarchical clustering
+ \nwith dissimilarity = 1-correlation", line=-0.5)
```

depr: *Principal Component* Returns Time Series

```
> # PC returns from rotation and scaled returns
> retsc <- apply(retp, 2, scale)
> retpca <- retsc %*% pcad$rotation
> # "x" matrix contains time series of PC returns
> dim(pcad$x)
> class(pcad$x)
> head(pcad$x[, 1:3], 3)
> # Convert PC matrix to xts and rescale to decimals
> retpca <- xts(pcad$x/100, order.by=zoo::index(retp))
```

```
> chart.CumReturns(
+   retpca[, 1:3], lwd=2, ylab="",
+   legend.loc="topright", main="")
> # Add title
> title(main="ETF cumulative returns", line=-1)
```

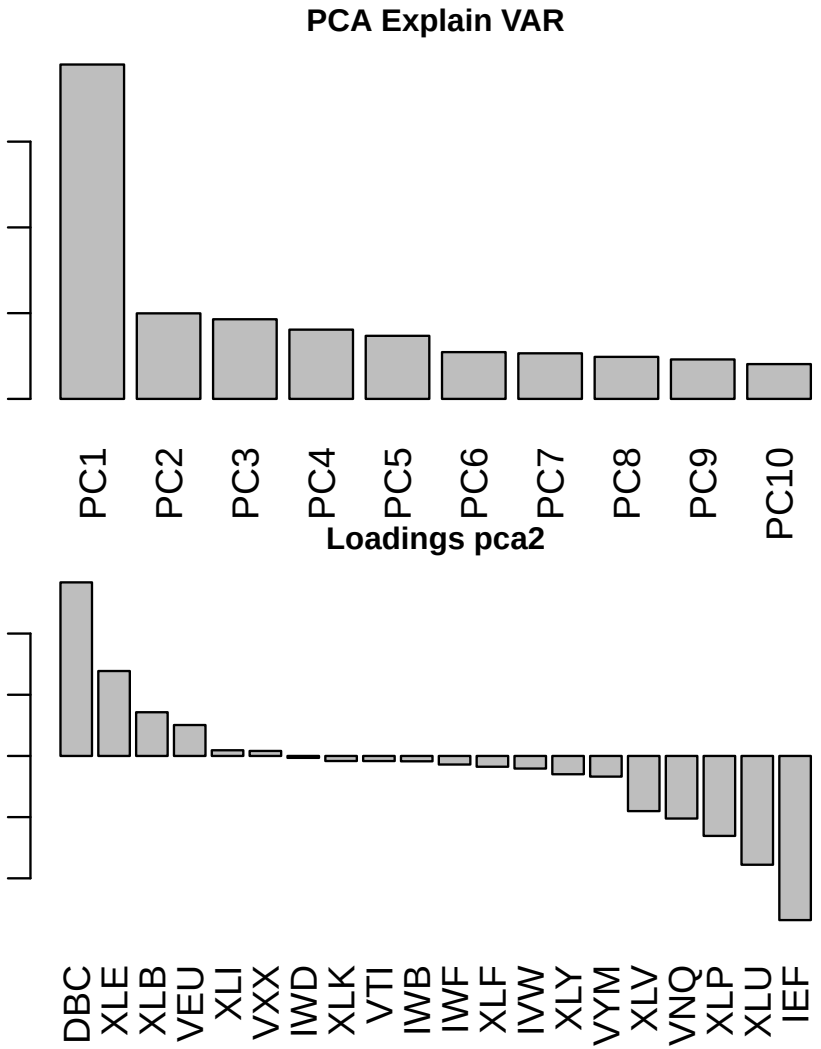


depr: *Principal Component* Returns Analysis

```
> # Calculate PC correlation matrix
> cormat <- cor(retpca)
> colnames(cormat) <- colnames(retpca)
> rownames(cormat) <- colnames(retpca)
> cormat[1:3, 1:3]
> table.CAPM(Ra=retpca[, 1:3], Rb=retp$VTI, scale=252)
```

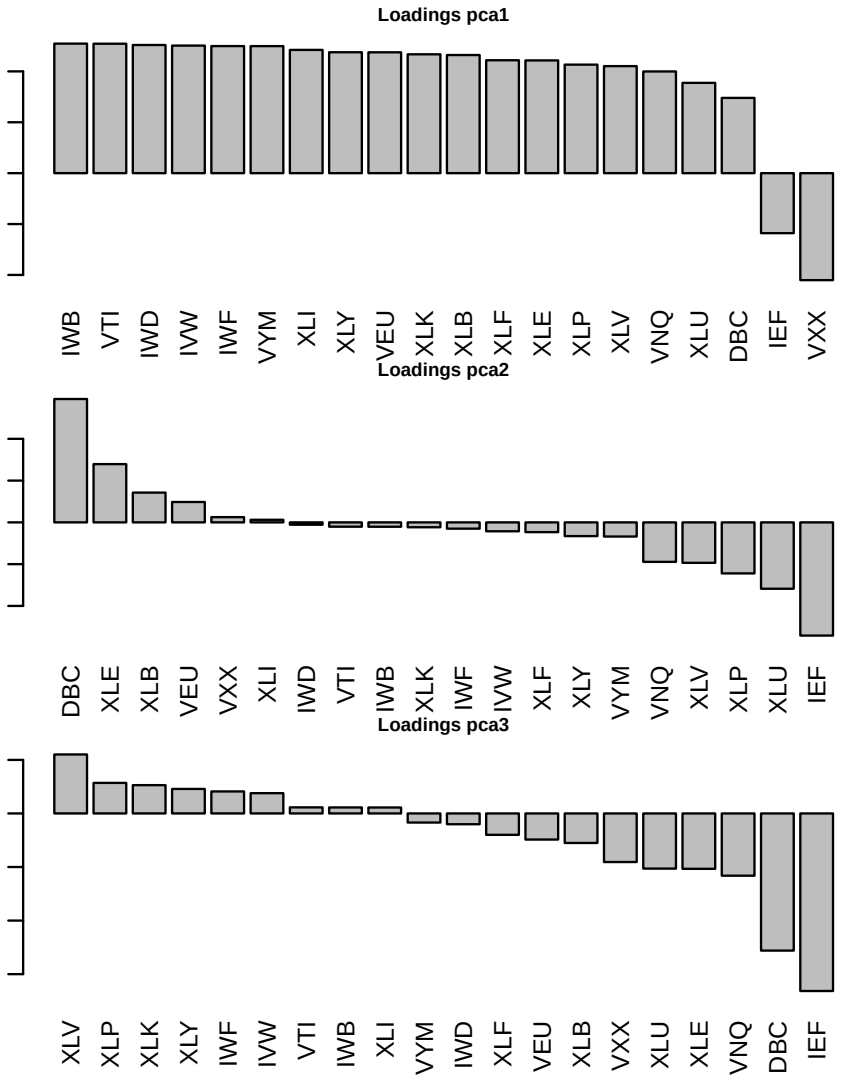
depr: Principal Component Analysis

```
> ### Perform principal component analysis PCA
> retp <- na.omit(rutils::etfenv$returns)
> pcad <- prcomp(retp, center=TRUE, scale=TRUE)
> barplot(pcad$sdev[1:10],
+   names.arg=colnames(pcad$rotation)[1:10],
+   las=3, ylab="STDEV", xlab="PCVec",
+   main="PCA Explain VAR")
> # Show first three principal component loadings
> head(pcad$rotation[,1:3], 3)
> # Permute second principal component loadings by size
> pca2 <- as.matrix(
+   pcad$rotation[order(pcad$rotation[, 2],
+   decreasing=TRUE), 2])
> colnames(pca2) <- "pca2"
> head(pca2, 3)
> # The option las=3 rotates the names.arg labels
> barplot(as.vector(pca2),
+   names.arg=rownames(pca2),
+   las=3, ylab="Loadings",
+   xlab="Symbol", main="Loadings pca2")
```



depr: Principal Component Vectors

```
> # Get list of principal component vectors
> pca_vecs <- lapply(1:3, function(ordern) {
+   pca_vec <- as.matrix(
+     pcad$rotation[
+     order(pcad$rotation[, ordern],
+     decreasing=TRUE), ordern])
+   colnames(pca_vec) <- paste0("pca", ordern)
+   pca_vec
+ })  ##  end lapply
> names(pca_vecs) <- c("pca1", "pca2", "pca3")
> # The option las=3 rotates the names.arg labels
> for (ordern in 1:3) {
+   barplot(as.vector(pca_vecs[[ordern]]),
+   names.arg=rownames(pca_vecs[[ordern]]),
+   las=3, xlab="", ylab="",
+   main=paste("Loadings",
+     colnames(pca_vecs[[ordern]])))
+ }  ##  end for
```



depr: Package *factorAnalytics*

The package *factorAnalytics* performs estimation and risk analysis of linear factor models for portfolio asset returns.

```
> library(factorAnalytics)  ##  Load package "factorAnalytics"
> # Get documentation for package "factorAnalytics"
> packageDescription("factorAnalytics")  ##  Get short description
> help(package="factorAnalytics")  ##  Load help page
```

```
> # List all objects in 'factorAnalytics'
> ls("package:factorAnalytics")
>
> # List all datasets in 'factorAnalytics' data(package='factorAnalytics')
>
> # Remove factorAnalytics from search path
> detach("package:factorAnalytics")
```

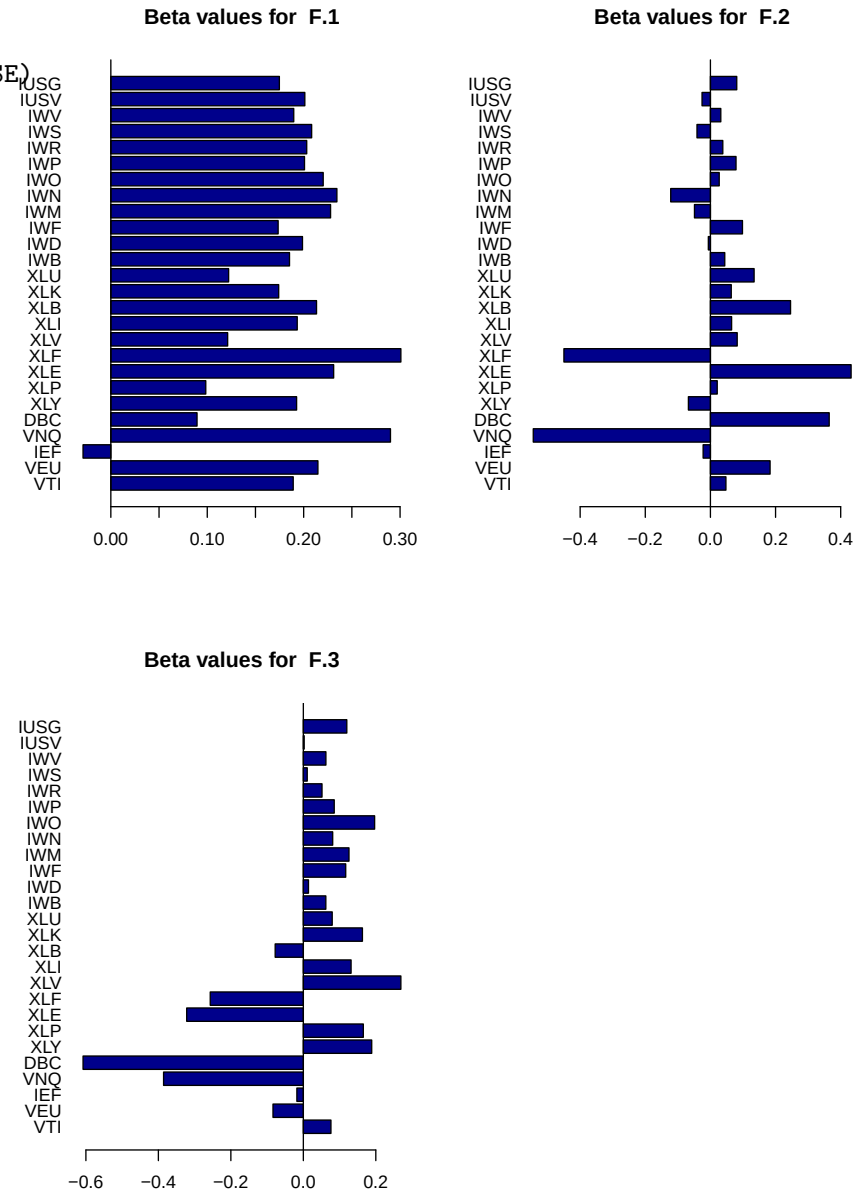
depr: Fitting Factor Models Using PCA

```
> library(factorAnalytics)
> # Fit a three-factor model using PCA
> factpca <- fitSfm(rutils::etfenv$returns, k=3)
> head(factpca$loadings, 3)  ##  Factor loadings
> # Factor realizations (time series)
> head(factpca$factors)
> # Residuals from regression
> factpca$residuals[1:3, 1:3]
```

```
> factpca$alpha  ##  Estimated alphas
> factpca$r2  ##  R-squared regression
> # Covariance matrix estimated by factor model
> factpca$Omega[1:3, 4:6]
```

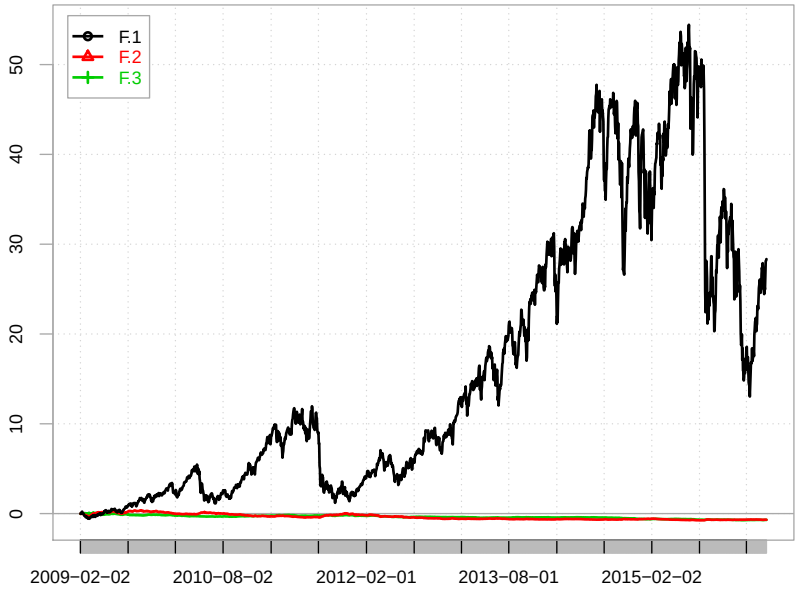
depr: Factor Loadings

```
> plot(factpca, which.plot.group=3, n.max=30, loop=FALSE)
> # ?plot.sfm
```



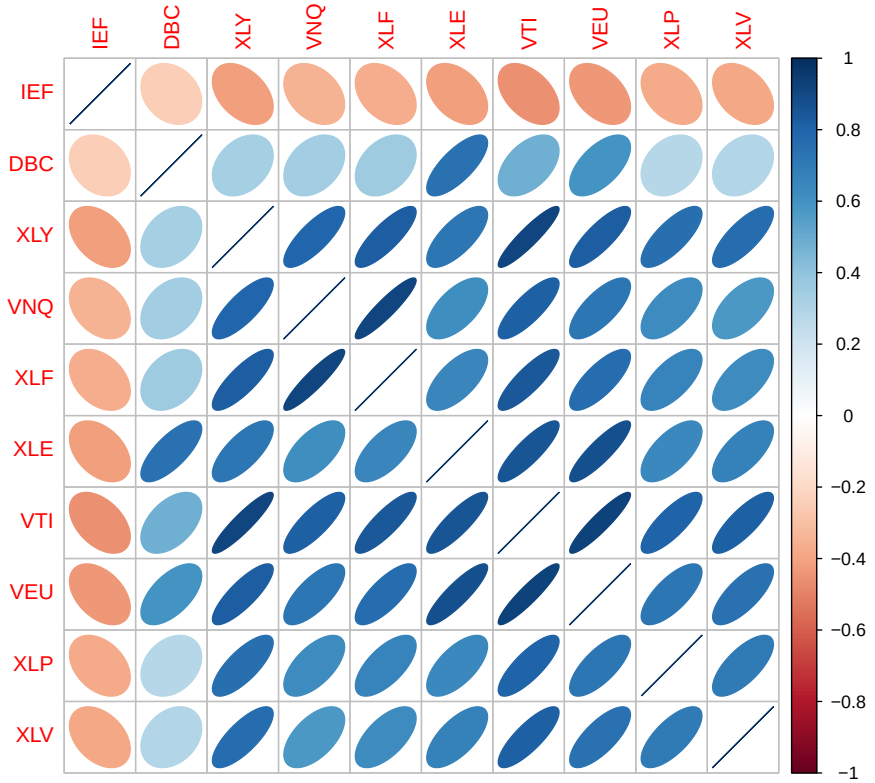
depr: Time Series of Factors

```
> library(PortfolioAnalytics)
> # Plot factor cumulative returns
> chart.CumReturns(factpca$factors,
+     lwd=2, ylab="", legend.loc="topleft", main="")
>
> # Plot time series of factor returns
> # Plot(factpca, which.plot.group=2,
> #   loop=FALSE)
```



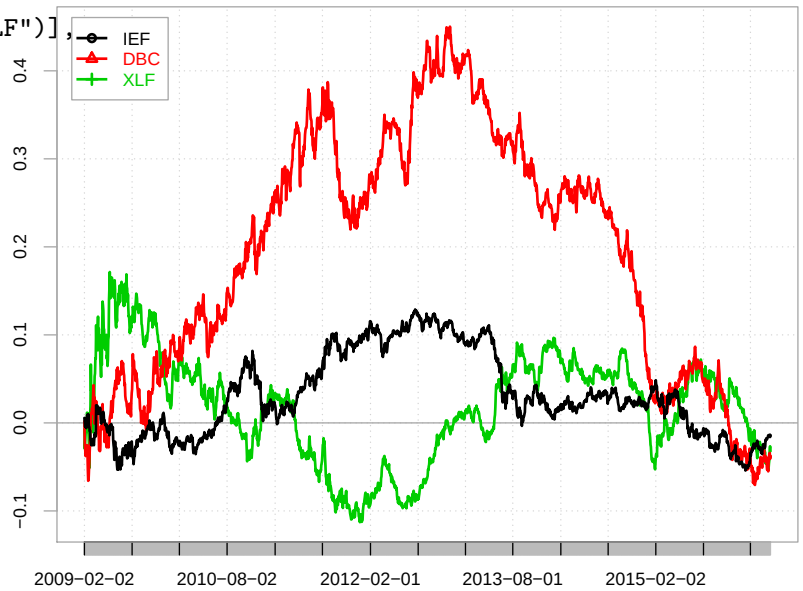
depr: Asset Correlations

```
> # Asset correlations "hclust" hierarchical clustering
> plot(factpca, which.plot.group=7, loop=FALSE,
+      order="hclust", method="ellipse")
```



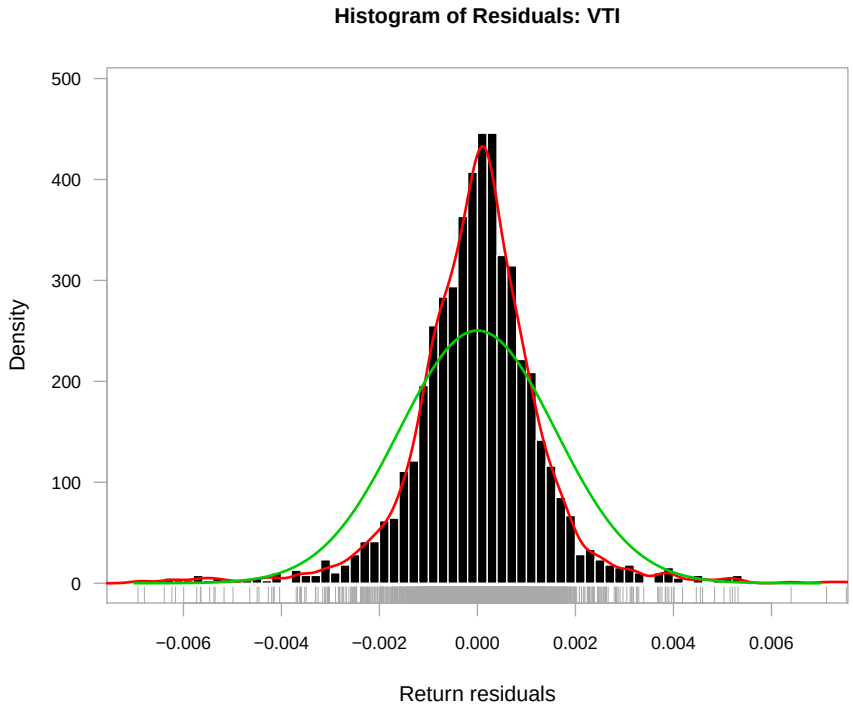
depr: Time Series of Residuals

```
> library(PortfolioAnalytics)
> # Plot residual cumulative returns
> chart.CumReturns(factpca$residuals[, c("IEF", "DBC", "XLF")],
+   lwd=2, ylab="", legend.loc="topleft", main="")
```



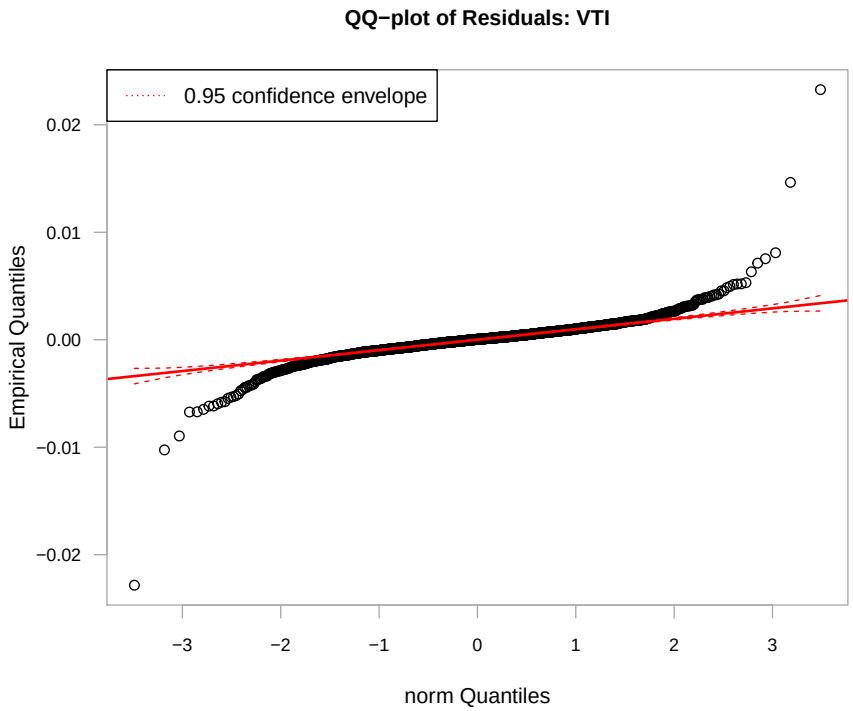
depr: Residual Returns Histogram

```
> library(PortfolioAnalytics)
> # Plot residual histogram with normal curve
> plot(factpca, asset.name="VTI",
+      which.plot.single=8,
+      plot.single=TRUE, loop=FALSE,
+      xlim=c(-0.007, 0.007))
```



depr: Residual Returns and the Q-Q Plot

```
> # Residual Q-Q plot
> plot(factpca, asset.name="VTI",
+      which.plot.single=9,
+      plot.single=TRUE, loop=FALSE)
```



depr: Autocorrelation of Residuals

```
> # SACF and PACF of residuals
> plot(factpca, asset.name="VTI",
+      which.plot.single=5,
+      plot.single=TRUE, loop=FALSE)
```